Handling Missing Data: Complete Case Analysis

Module 35 Study Notes • Feature Engineering

1. Introduction to Missing Data

Machine Learning algorithms are inherently mathematical. Because they rely on equations, the vast majority of algorithms in Scikit-Learn will instantly crash if you feed them a dataset containing missing values (NaN, Null). As a Data Scientist, it is your absolute responsibility to address missing data before initiating the training phase.

Broadly speaking, you have two overarching strategies when encountering missing data:

- **Strategy 1: Remove the Data.** Delete the specific row (Observation) or column (Variable) containing the missing value.
- **Strategy 2: Impute the Data.** Calculate a logical replacement (like the mean, median, or a machine-learning prediction) and fill in the blank.

2. Complete Case Analysis (CCA)

This module focuses exclusively on the first strategy. **Complete Case Analysis (CCA)**, also known as "Listwise Deletion," is the practice of discarding any observation (row) that contains a missing value in *any* of its columns.

The term "Complete Case" signifies that your final training dataset will consist only of "complete cases"—rows where every single feature has a recorded value.

3. The Golden Rule of CCA: MCAR

You cannot simply delete rows blindly. CCA is statistically valid **only** under a very strict assumption: The data must be **Missing Completely At Random (MCAR)**.

What is MCAR?

It means there is no underlying pattern to why the data is missing. The probability of a value being missing is completely independent of the actual unseen value itself, and independent of any other columns in the dataset. It's as if someone threw darts at the dataset blindly to delete values.

> **The Danger of Breaking the Rule:** If the data is *not* MCAR (e.g., in a survey, high-income individuals systematically refuse to disclose their salary), deleting those rows will destroy the shape of your dataset. You will accidentally purge a specific demographic, severely biasing your model and ruining its predictive power.

4. Implementation Workflow & Checks

Because ensuring the data shape doesn't change is so vital, implementing CCA is a multi-step verification process.

Step 1: The 5% Threshold

As an industry rule of thumb, you should only consider CCA if the total proportion of missing data in a given column is **less than 5%**. If you are missing 30% of the data, CCA will delete too much of your dataset.

```
# Identify columns with less than 5% missing data (but greater than 0%)
cols_for_cca = [col for col in df.columns if df[col].isnull().mean() < 0.05 and df[col].isnull().mean() > 0]
```

Step 2: Drop the Rows

Use Pandas' built-in dropna function to execute the deletion.

```
# Drop rows where any of the specified columns contain a NaN
df_cca = df.dropna(subset=cols_for_cca)
```

Step 3: Verify the Distribution (Numerical Data)

You must plot the Probability Density Function (PDF) or Histogram of the column *before* CCA and overlay it with the plot *after* CCA. If the MCAR assumption holds true, the two bell curves should almost perfectly overlap. If the new curve shifts left or right, your data was not MCAR, and CCA has biased your dataset.

```
import matplotlib.pyplot as plt
import seaborn as sns
```

```
fig = plt.figure()
ax = fig.add_subplot(111)

# original data (red)
df['Age'].plot.density(color='red')

# data after cca (blue)
df_cca['Age'].plot.density(color='blue')
```

Step 4: Verify the Ratios (Categorical Data)

For categorical data, compare the percentage ratio of each category before and after. If "High School" represented 40% of the dataset before CCA, it must still represent approximately 40% of the dataset after CCA.

```
# Calculate ratio before CCA
temp = df['Education_Level'].value_counts() / len(df)

# Calculate ratio after CCA
temp_cca = df_cca['Education_Level'].value_counts() / len(df_cca)
```

5. The Fatal Flaw of CCA in Production

Even if your data perfectly passes the 5% threshold and the MCAR checks, CCA has one massive, fatal flaw when transitioning from a Jupyter Notebook to a real-world web server:

Your model does not know how to handle blanks.

Because you deleted all the blanks during training, your model was never taught what to do when it encounters one. When your model is deployed in production, and a user inevitably leaves a field blank on the website form, your server will crash because the model cannot process a `NaN`. This is why, despite its simplicity, Data Scientists rarely rely entirely on Complete Case Analysis, preferring **Imputation** (which we will cover next).